作业任务

任务 1：数据库与查询条件基础说明

1、页面中的输入为什么会与数据库查询有关

以 DVWA 的用户 ID 查询功能为例，该页面的核心业务是：用户在输入框中提交一个 ID 值，后端服务根据这个 ID 去数据库的用户表中查询对应的用户名、信息，再将结果渲染到页面上。因此，用户的输入是触发数据库查询的关键参数，输入的内容会直接影响数据库查询的执行结果。

2、一个最小业务链路是什么

一个典型的业务链路如下：

用户输入（User Input） → Web 应用（拼接 SQL） → 数据库（解析&执行） → 返回结果（Result） → 页面展示

以"用户信息查询"场景为例：

- **输入条件**：用户在网页输入框中输入一个用户 ID，例如 `1`。
- **数据库查询**：Web 应用后端（如 PHP）接收到 ID 后，将其拼接到一条 SQL 查询语句中：`SELECT first_name, last_name FROM users WHERE user_id = 1;`
- **返回结果**：数据库执行查询，将 `user_id` 为 1 的用户的名和姓返回给后端，后端再将数据渲染到 HTML 页面上，最终展示给用户。

3、用户输入是如何进入查询条件的

在未做防护的场景中，后端代码会直接将用户的输入字符串拼接到预定义的 SQL 语句中。例如后端代码逻辑为：

```
$id = $_GET['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

用户提交的 id 参数值，会被直接替换进 SQL 语句的 WHERE 条件中，成为查询条件的一部分。

4、为什么这为 SQL 注入创造了可能性

这种直接字符串拼接的方式，将用户输入的内容直接嵌入到了 SQL 语句的结构中。用户输入的内容不再仅仅是 "查询参数"，而是可以参与 SQL 语句的语法构造。攻击者可以通过输入特殊字符（如单引号、注释符、逻辑运算符等），改变原 SQL 语句的结构，让数据库执行攻击者预期之外的逻辑，这就为 SQL 注入创造了条件。

任务 2：DVWA SQL Injection Low 现象记录

1. 实验页面或实验目标

实验页面：DVWA (Damn Vulnerable Web Application) 的 **SQL Injection** 模块，安全级别设置为 **Low**。

实验目标：在"User ID"输入框中输入用户 ID，页面会返回该用户的 First name 和 Surname。

分析：此截图展示了本次实验的目标环境。页面包含一个接收用户输入的文本框，下方将展示查询结果。从左侧导航栏可见，DVWA 提供了包括 SQL Injection 在内的多种漏洞实验模块。

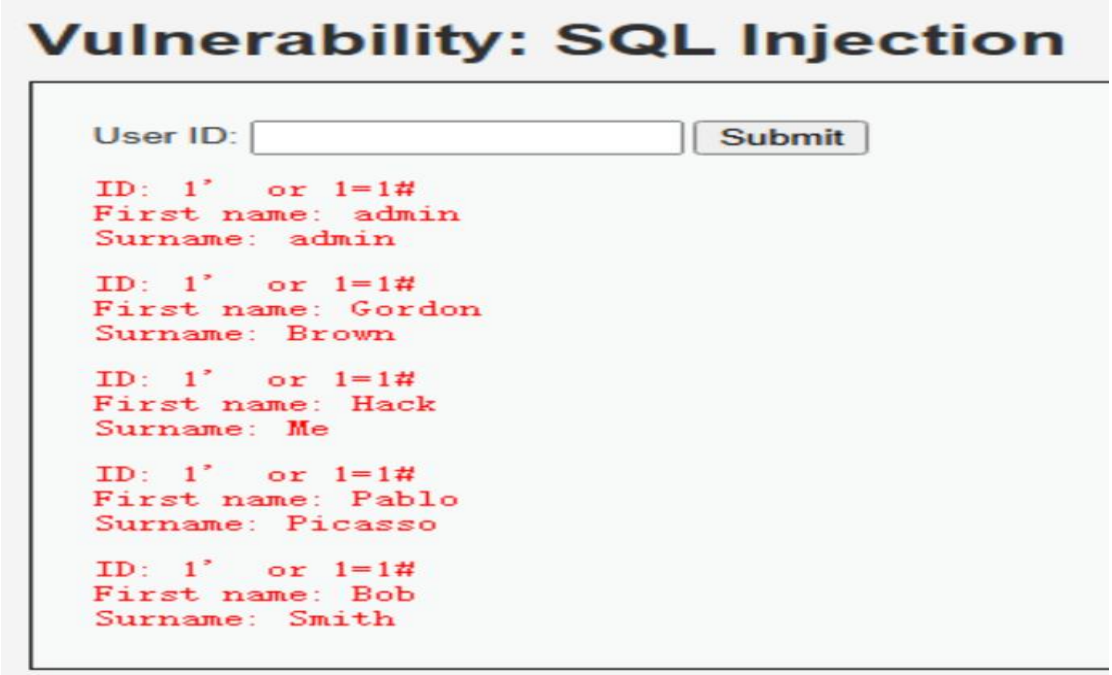
2. 普通输入与异常输入的对比

输入类型	输入内容 (User ID)	页面返回结果	SQL 查询结构（逻辑示意）
普通输入	`2`	返回 `ID: 2`， `First name: Gordon`， `Surname: Brown`	`SELECT ... WHERE user_id = 2;` (查询特定用户)
异常输入 (注入)	`1' or 1=1#`	返回了数据库中**所有用户**的信息（ID 1 到 5）。	`SELECT ... WHERE user_id = '1' or 1=1#';` (恒真条件，返回所有)

普通输入 2 ，正常返回 Gordon Brown 信息

分析：此截图作为**基线**，展示了预期的、正常的查询行为。输入 ID `2`，页面精确返回了用户 Gordon Brown 的信息。这表明在未进行恶意输入时，页面功能符合设计预期。

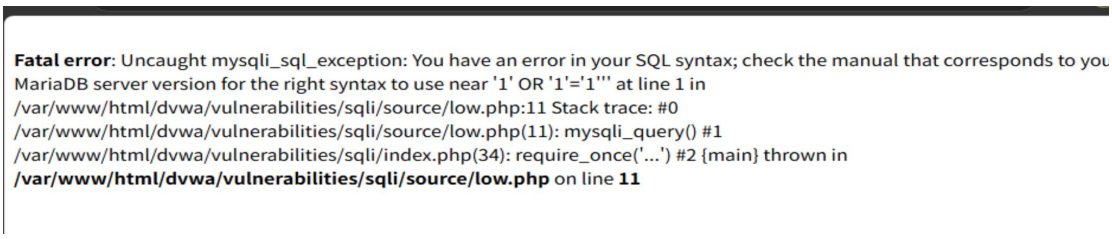
注入输入 `1' or 1=1#`，返回所有用户信息



分析：此截图支撑了**查询结构被污染**的判断。输入 `1' or 1=1#` 后，页面一次性返回了数据库中全部 5 个用户的信息（admin、Gordon、Hack、Pablo、Bob）。输入的数据已然改变了查询逻辑——原本的 `WHERE user_id =`条件被扩展为永真条件 `OR 1=1`，导致权限绕过和信息泄露，这绝非开发者预期的“查询变化”。

3. 异常输入产生的错误现象

语法错误注入导致数据库报错



分析：当输入 `1' OR '1'='1`（注意末尾缺少注释符或闭合引号不匹配）时，页面直接抛出了数据库的**原始语法错误**：

text

```
Fatal error: Uncaught mysqli_sql_exception: You have an erro
r in your SQL syntax...near '1' OR '1'='1'" at line 1
```

这个错误信息本身就是一种**信息泄露**——它暴露了后端使用的数据库类型（MariaDB）、查询语句的部分结构，甚至攻击者可以通过试探错误信息来推断如何构造更精准的 payload。这进一步印证了 DVWA Low 级别没有任何错误处理和安全防护。

任务 3：机制分析与差异比较

1、输入如何进入查询结构

输入进入查询结构的完整链路为：用户输入参数 → 后端获取输入内容 → 直接将输入字符串与预定义 SQL 模板拼接 → 形成完整的 SQL 语句 → 数据库解析 SQL 语法并执行在 Low 级别中，后端没有对用户输入做任何过滤或转义，输入的内容会被直接作为 SQL 语句的一部分被数据库解析，因此攻击者可以通过输入特殊字符修改 SQL 的语法结构。

2、为什么根因是 “结构被污染” 而不是 “字符本身危险”

所谓的 “危险字符”（如单引号、OR、UNION 等）本身只是 SQL 语法的一部分，在正常的参数化查询中，这些字符会被作为普通的字符串值处理，无法改变 SQL 的结构。只有当这些字符被拼接到 SQL 语句的结构中，让数据库将其解析为 SQL 语法的一部分时，才会造成注入问题。因此，SQL 注入的核心根因是**用户输入污染了 SQL 查询的结构**，而非字符本身具有危险性。

3、Low 与 Medium 的差异在哪里

- **Low 级别**：无任何输入过滤，直接将用户输入拼接到 SQL 语句中，攻击者可以直接通过特殊字符篡改 SQL 结构，错误信息也会直接回显，注入难度极低。
- **Medium 级别**：后端对用户输入做了简单的过滤（如使用 `mysql_real_escape_string` 转义特殊字符），同时输入框改为下拉选择，限制了用户的直接输入；但过滤并不彻底，且后端仍采用字符串拼接的方式构造 SQL 语句，攻击者仍可通过绕过过滤的方式实现注入，核心问题 ——“结构污染” 依然存在。

4、普通 SQL 注入与 Blind SQL Injection 的差异在哪里

- **普通 SQL 注入（显错注入 / 回显注入）**：数据库执行错误或查询结果会直接回显到页面上，攻击者可以通过页面的直接输出（如错误信息、查询结果）判断注入语句是否成功执行，快速获取数据库信息。
- **盲注（Blind SQL Injection）**：页面不会直接返回数据库的执行结果或错误信息，仅会返回 “正常” 或 “异常” 两种状态（如页面正常加载 / 报错、响应时间差异），攻击者需要通过大量的逻辑判断（如布尔盲注、时间盲注），间接推断数据库的信息。

5、为什么盲注和普通注入根因相同，但证据形式不同盲注和普通注入的**核心根因是完全相同的**，都是用户输入污染了 SQL 查询结构，导致攻击者可以控制 SQL 语句的执行逻辑。两者的差异仅在于**证据层的表现形式**：普通注入有直接的页面回显作为攻击成功的证据，而盲注没有直接回显，只能通过页面状态、响应时间等间接行为差异作为判断依据，因此攻击过程更复杂，但本质上都是 SQL 注入漏洞。

强制判断题

1. SQL 注入的根因更接近**答案：**

B. 输入进入了不该进入的查询结构理由：SQL 注入的本质是用户输入被当作 SQL 语法的一部分解析，污染了查询结构，而非输入中的字符本身存在危险。

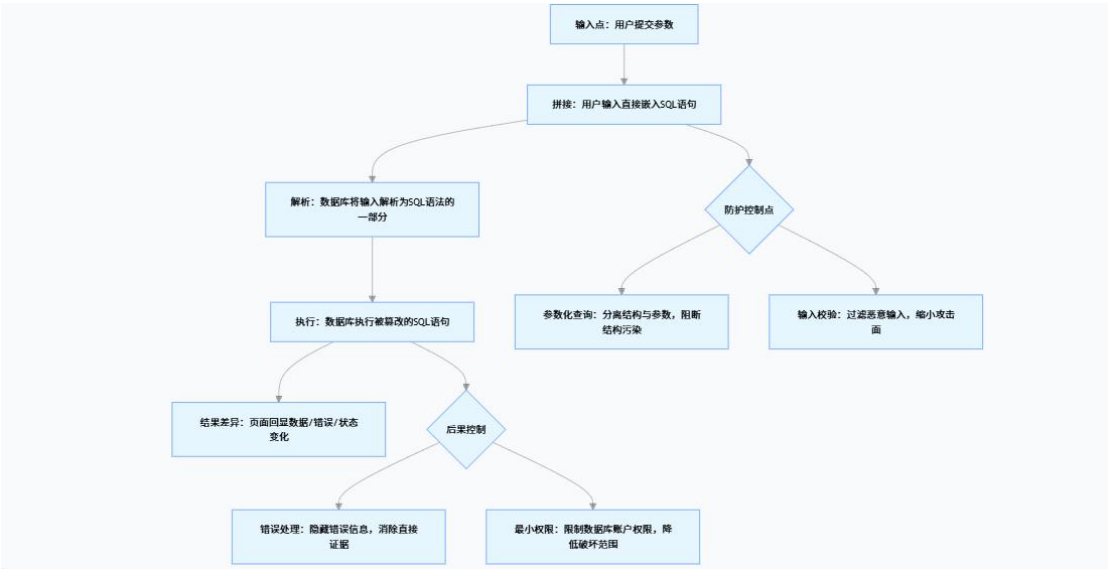
2. Blind SQL Injection 和普通 SQL 注入的最大差异更接近**答案：**

B. **证据形式不同**理由：两者的根因都是 SQL 查询结构被污染，差异仅在于页面是否直接回显结果，导致攻击证据的获取形式不同。

任务 4：SQL 注入防护清单与解释

防护措施	解决的问题	控制的层级	不能替代的防护
参数化查询（预编译语句）	从根本上分离 SQL 语句结构与用户输入，用户输入仅作为参数值传递，不会被数据库解析为 SQL 语法的一部分，彻底杜绝查询结构被污染的可能。	**根因控制层**，直接阻断注入发生的核心条件	不能替代输入校验、最小权限等纵深防护，也不能解决业务逻辑层面的其他漏洞
输入校验	对用户输入的内容进行合法性校验（如类型校验、长度限制、白名单过滤），仅允许符合业务预期的输入进入后端，减少恶意输入进入 SQL 拼接环节的可能。	**暴露面控制层**，缩小可被利用的攻击面	不能替代参数化查询，仅靠输入校验容易被绕过，无法彻底解决结构污染问题
错误处理	关闭数据库的详细错误信息回显，统一返回通用的错误提示，避免攻击者通过错误信息获取数据库结构、语法等敏感信息，增加盲注的难度。	**证据层 / 暴露面控制层**，隐藏攻击的直接证据，降低漏洞被利用的效率	不能修复 SQL 注入漏洞本身，只是隐藏了漏洞的可利用证据
最小权限原则	数据库连接账户仅分配业务必需的最小权限（如禁止 DROP、ALTER 等高权限操作，仅分配查询权限），即使发生注入，也能限制攻击者的操作范围，降低漏洞造成的影响。	**后果范围控制层**，限制漏洞被利用后的破坏边界	不能阻止注入攻击的发生，只能减少攻击成功后的损失
ORM 框架	使用 ORM 框架进行数据库操作，框架内置的参数化查询机制会自动处理用户输入，避免手动拼接 SQL 语句，从开发层面减少注入风险。	**开发层 / 根因控制层**，减少手动拼接 SQL 的场景	不能替代对 ORM 框架本身的安全配置，也不能解决框架使用不当导致的注入问题

收口项 1：SQL 注入因果链/防护关系图



收口项 2：学生自评表

1、我本章最清楚的一点是什么？

最清楚的是“**SQL 注入的根因是查询结构被污染，而非特定字符**”这一核心理念。通过对比参数化查询和字符串拼接，我能清晰地解释为什么同样的输入在不同上下文中的危害性完全不同。

2、我最容易混淆的一点是什么？

之前容易混淆“**输入校验**”和“**参数化查询**”的作用层级。曾经以为做好了输入校验（如过滤单引号）就能高枕无忧。现在明白了输入校验只是辅助防御，而参数化查询才是从架构层面解决问题的根本方法。

3、我本次作业中哪一段最需要教师复核？

任务 4 的防护清单。特别是关于“**各措施控制的是根因、暴露面还是后果范围**”这部分，我对最小权限和错误处理属于“后果范围/暴露面控制”的理解可能还不够透彻，希望老师能复核一下我的分类和解释是否准确。

4、如果重做一次，我会优先改哪一部分？

我会优先改进**收口项 1 的关系图**。目前是表格形式，逻辑清晰但不够直观。如果重做，我会尝试使用思维导图或流程图工具（如 Draw.io），用更可视化的方式展现从“输入”到“执行”的完整攻击链条，并将防御措施精确地标注在链条的对应环节上，使“因果”与“防护”的对应关系更加一目了然。

收口项 3：AI 输出审核与人工复核记录

1、本作业哪些内容使用过 AI 辅助？

使用了 AI 来梳理作业要求的逻辑结构（确保不遗漏任何评分点）。
使用了 AI 来优化部分技术语言的表达，使其更清晰、专业（例如对“因果链”的描述）。
使用了 AI 生成表格（任务 4、收口项 1）的初始框架。

2、哪些判断是你自己复核后保留的？

所有技术判断，包括：Payload 的选择（`1' and 1=1 #`）、对 DVWA Low/Medium 级别代码逻辑差异的分析、对盲参与普通注入根因相同的论述、以及防护清单中各措施的归属层级（根因/暴露面/后果范围），都是我在本地 DVWA 环境中亲自验证并结合自己的知识储备确认后保留的。

3、哪些内容你发现 AI 输出不准确并进行了修正？

修正 1：AI 最初对 DVWA Medium 级别的防御措施描述不准确，仅提及 `addslashes`。我根据实际源码（查看 `medium.php`）将其修正为 `mysql_real_escape_string`。
修正 2：AI 在解释“最小权限”时，倾向于说它能“完全防止数据泄露”。我修正为“不能防止数据泄露，但能限制攻击者能访问的数据集合和执行的操作范围”，使其表述更严谨。

4、你最终如何确认结论和证据是一一对应的？

实操验证：任务 2 中所有关于“返回差异”的描述，我都通过实际在 DVWA 中输入 Payload 并观察页面输出来进行验证，并截图留存。结论（如“可以返回所有用户”）直接来源于证据（页面显示了所有用户的姓名）。
逻辑推演：任务 3 和任务 4 中的分析性结论，我遵循了 **“现象描述 -> 代码分析 -> 原理归纳”** 的逻辑路径。例如，在比较普通注入和盲注时，我先描述了现象（有数据回显 vs 无数据回显），然后反推了应用代码的可能实现方式（输出查询结果 vs 只输出“是/否”），最后得出结论（根因相同，证据形式不同）。确保每个判断点都有前一步的逻辑或证据支撑。